# PostgreSQL Scaling Lab

Brief

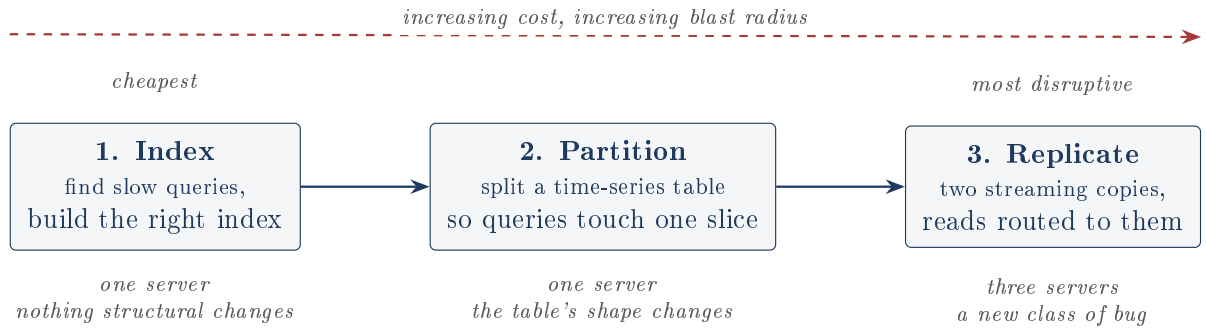
Three techniques for making one relational database do more, in escalating order of cost: index it, partition it, replicate it. What each one buys, what each one costs, and which of this project's published numbers survive a re-measurement.
A companion Deep Dive covers the same ground exhaustively, with every term defined at first use. This document assumes you can read a `SELECT` and want the shape of the thing, the trade-offs, and the honest state of the evidence.

---

## 1 The premise

A company runs on PostgreSQL. Queries are slowing down. The CTO has heard NoSQL is "infinitely scalable" and wants to migrate. The counter-argument this project makes is not rhetorical: it works through what the standard toolbox actually delivers, measures it, and reports the cases where it does not help.

Three parts, three independent Docker Compose stacks, run one at a time:




**What is the author's, and what was handed to him**

The assignment briefs, the `init.sql` schemas, the baseline app (`part_3/app/app.py`), the test suite, and the prebuilt Go load generator are course-provided. The author's work is the three solution files, the entire part 3 replication configuration, the routing app `solution.py`, and the analysis. The code volume is small; the reasoning is the deliverable. This is stated plainly in the project README, which is the right call.

## 2 Part 1: indexing

### 2.1 The mechanism

A table is a pile of 8 KB blocks in no particular order. With no index, finding one row means reading every block and discarding almost everything. An index is a separate sorted structure mapping values to row locations, so a lookup becomes a short descent instead of a full sweep.

Measured on this project's own 100,000-row `users` table: **2,494 blocks → 4 blocks**, and 99,999 wasted row inspections → 0. That is the entire value proposition, and it is reproduced in `docs/screenshots/run-output.txt` and again independently while writing these documents.

## 2.2 The real lesson: an index is three decisions, not one

The transferable idea in part 1 is not "add indexes". It is that `CREATE INDEX` is a choice of column, index *type*, and *operator class*, and getting the last one wrong builds an index the planner will never use.

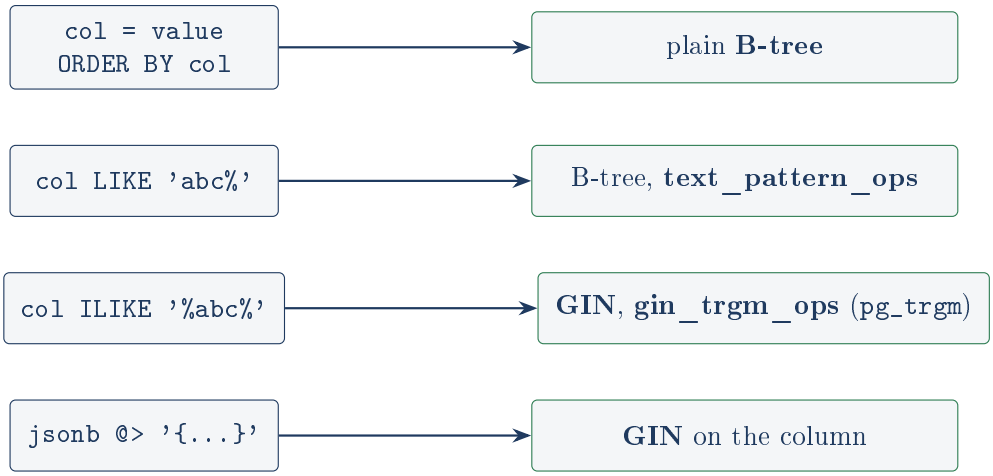



**Figure 1:** The decision `part_1/solution.sql` encodes in seven lines.

Why each one is forced:

- `text_pattern_ops`: a default text B-tree sorts by locale rules, so PostgreSQL cannot prove that everything starting `'user1'` is contiguous in the index, and refuses to use it for `LIKE 'user1%'`. Sorting by raw character value restores that guarantee. The trade: the index can no longer serve `ORDER BY` in locale order. Here nothing needs that, so it is a good trade; the project does not mention that it is a trade at all.
- **Trigram GIN**: for `'%abc%'` the match can start anywhere, so no sort order helps. `pg_trgm` chops strings into three-character slices and GIN maps each slice to the many rows containing it. It returns *candidates* that must be re-checked, which is exactly why it disappoints (310 ms → 180 ms in the project's own reporting, correctly explained there).
- **JSONB GIN**: same structure, indexing every key and value path so `@>` containment can be answered from the index.

## 2.3 Where the reporting does not survive contact

All of the following were checked by loading this repository's own `init.sql` and `solution.sql` into a real PostgreSQL and reading the plans.

**The load generator is not a black box**

The README lists it as a challenge: prebuilt Go binaries, no source, "I could not change the query mix or read what it was doing." The response (turn on `auto_explain` and make the database report its own plans) is excellent and right regardless. But Go embeds string literals in the binary, and one command recovers the full 14-query mix:

```
grep -aoE "SELECT [ -~]{15,200}" load_tester_amd64 | grep -iE "FROM (users|orders)"
```

"Could not change" stands. "Could not read" does not. This matters because knowing the mix lets you say which index serves which real query, and doing that turns up two claims the

real queries contradict.

**The status index is credited to a query that cannot use it**

`INDEXING_ANSWERS.md` calls `status` "low selectivity (only 2 values)" and credits `idx_users_status` with a 30% gain. Three problems, all measured:

- Distinct-value count is not selectivity. The real split is **99,000 active to 1,000 inactive**. `'inactive'` is a 1% predicate and the index is excellent for it (59 ms → 10.6 ms). `'active'` is 99% and nothing can help.
- The load tester's only status filter is `status = 'active'`, the 99% case. Measured: the planner correctly ignores the index and scans sequentially. It cannot be responsible for the claimed gain.
- The index *does* earn its keep, via a query the notes never mention: `SELECT status, COUNT(*) ... GROUP BY status` is served by an index-only scan over 696 KB instead of an 85 MB heap.

Keep the index, replace the story.

**"Indexes don't help `COUNT(*)`" is wrong, and this project disproves it**

The visibility argument in the notes is right (PostgreSQL must check each row's visibility, so there is no free counter). The conclusion is not. Measured: `SELECT COUNT(*) FROM users` runs as an **Index Only Scan on** `idx_users_created_desc` touching **87 blocks**; with index-only scans disabled it sequentially scans **2,494**. PostgreSQL walks the smallest covering index rather than the heap, because counting needs to visit rows, not read columns. The author built that index for sorting and accelerated `COUNT(*)` by roughly 29x without noticing.

**An index can lose, and here one does**

JSONB containment, re-measured at two selectivities on this project's data:

| Predicate matches | Without GIN | With GIN | |
|---|---|---|---|
| 50,000 rows (50%) | 220 ms | 299 ms | **index loses** |
| 16,666 rows (17%) | 148 ms | 82 ms | index wins |

An index is a bet that most rows are irrelevant. At 50% the bet loses: you pay for the index descent and visit half the heap anyway, in random order, when an ordered sweep was cheaper. The planner still *chose* the index in the 50% case and lost, which is a genuine misestimate on a JSONB column where statistics are weak. Worth being able to say out loud.
The real load-tester JSONB query is worse: it pairs `@>` with `metadata->>'last_login' > '2023-01-01'`, and only the containment half is indexable. Measured: GIN produces 50,000 candidates, the second filter discards all but 646.

**The slowest real query is not fixable by any of the seven indexes**

`SELECT DISTINCT u.* FROM users u JOIN orders o ON u.id = o.user_id WHERE u.full_name ILIKE '%User%' AND o.amount > 500` measures **2,932 ms** with every index in place. Neither the trigram index nor the amount index is used, and correctly so: the predicates match all 100,000 users and half the 500,000 orders. The assignment's own brief anticipates this ("consider if some queries are naturally slow"). This one is. *(Inferred: this is very likely the query behind the README's "4,500ms → 1,800ms" row; the shape and*

*magnitude match, but the README does not name it.)*

Every one of the seven indexes is used by at least one real query, so the index set as a whole is justified. What is not justified is some of the attribution.

# 3 Part 2: partitioning

## 3.1 The mechanism

Indexes make finding a row cheap; they do not make a table smaller. Partitioning splits one logical table into many physical ones by a rule on a column (here `event_time`). Writes are routed automatically. On read, the planner proves which partitions cannot contain a match and skips them entirely. That is **partition pruning**, and it is the whole point.

The strategy in `part_2/solution.sql` is defensible: monthly partitions for cold 2024 history, weekly partitions for hot recent data, plus a default catch-all. Coarse where it does not matter, fine where it does.

## 3.2 What reproduces, and what does not

**The claim that survives**

Pruning works and is visible in the plan: `Subplans Removed: 15`. Sixteen partitions, exactly one scanned (`events_202403`) for a March query. This is reproducible and it is what partitioning actually buys.

**The 9,154 → 975 block headline does not reproduce**

The README's flagship part 2 number is "monthly analysis dropped from 9,154 buffer blocks read to 975". Re-measured on this repository's own data with both tables freshly `VACUUM ANALYZE`d: **236 blocks → 791 blocks**. Partitioning read 3.3x *more*.

Pruning is not at fault; it works. The *baseline* is the issue. `init.sql` ships `idx_events_time_type` on `(event_time, event_type)`, and the monthly query needs only those two columns. So a properly analyzed single table answers it with an **Index Only Scan** over 236 narrow index blocks and never touches the heap. After partitioning, all of March *is* the partition, so a sequential scan of its 788 wide heap blocks wins on cost.

*(Inferred: 9,154 is the same order as a full sequential scan of the `events` heap, measured here at 10,910 blocks, and 975 is close to the 788-block March partition plus its index. The most likely explanation is a baseline measured on a freshly started container before `ANALYZE` ran, leaving the planner without the statistics to pick the index-only scan. The number is probably a real reading of a real plan; it just compares a no-statistics baseline against a partitioned table rather than isolating partitioning.)*

The narrower claim is stronger because it reproduces: *sixteen partitions pruned to one.*

**The negative result is real; its stated reason belongs to a different query**

The README says user-activity queries got 70% slower "because they filter on `user_id`, not the partition key, so every partition gets probed." That mechanism is exactly right. It just is not the benchmarked query.

**The benchmark row** (part 2, query 3) filters on `event_time >= NOW() - INTERVAL '30 days'`, which *is* the partition key. Re-measured: it did get slower (186 ms → 520 ms) and pruning did happen. It is slower because every recent row now sits in one default partition that gets sequentially scanned, plus runtime pruning overhead.

**The query the explanation describes** is `get_user_activity` from part 3's `solution.py` (`WHERE user_id = %s`). Re-measured against the part 2 tables, it behaves precisely as claimed: **55 blocks in 1 bitmap scan** on the single table versus **80 blocks across 13 index scans**, one per partition, because nothing lets the planner prune.
Both slowdowns are real and both are worth discussing. The labels are crossed. That is a one-sentence fix, but an interviewer reading `part_2/README.md` would hit it.

### The migration silently duplicates ids, which is worse than the README says

The README notes that `INSERT INTO events_partitioned SELECT * FROM events` never advances the sequence, "so fresh inserts would collide". Verified, and the reality is nastier:

- `events` has a primary key. `events_partitioned` has **none** at all: PostgreSQL requires a partitioned table's primary key to include the partition key, so `PRIMARY KEY (id)` is illegal and only `PRIMARY KEY (id, event_time)` was available. The migration quietly drops a uniqueness guarantee.
- Sequence after migration: `last_value` = **1**, `max(id)` = **600,000**.
- So there is no collision, because a collision needs a constraint to violate. An insert **silently succeeds with a duplicate**. Verified: the next insert got `id = 1`, and `id = 1` then existed twice.

Silent is worse than loud. The fix needs `setval()` *and* an explicit decision about the composite key.

### The interesting half of the strategy is currently inert

`solution.sql` hardcodes partitions for 2024 and for three weeks of November 2025. `init.sql` generates recent data relative to `NOW()`. They only align if you run the project in November 2025. Verified today: `events_default` holds **100,000 rows** (every recent event), and the three weekly "hot data" partitions hold **0 each**.
The 2024 monthly pruning still works, so the headline mechanism is unaffected. But the mixed-granularity design, which is the part worth defending, demonstrates nothing when run today. The README already proposes the fix (compute boundaries from the data with a `DO` block, or use `pg_partman`).
Two smaller data bugs, both the course's: `part_2/README.md` says recent data spans 7 days while the generator uses 14, and because it subtracts days then adds hours and minutes, **3,717 rows land in the future**.

### One benchmark measures nothing at all

Query 4 filters `user_id = 12345`, but the generator produces ids 1 to 10,001. **That user does not exist.** Verified: 0 rows, before and after. This explains the "∼0%" row in `PARTITIONING_ANSWERS.md` innocently and completely, but it is reported as a result rather than recognised as vacuous.

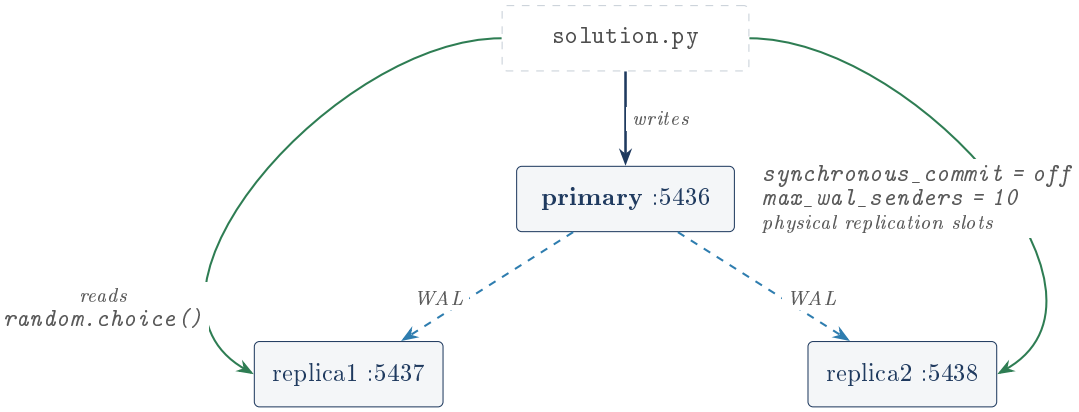
### The honest headline for part 2

Partitioning did not make this workload faster. It made the search space smaller and the access pattern explicit, and it made one query class substantially worse. It is a reorganization, not a speedup, and it can lose to a well-chosen covering index on a single table. Here it does.

# 4 Part 3: read replicas

## 4.1 The mechanism

PostgreSQL writes every change to a sequential log (the WAL) before touching data, for crash safety. Replication reuses that: ship the log to another server, replay it, and you have an identical server. A replica with `hot_standby = on` serves read-only queries while it replays.




Two pieces of the setup are genuinely well done and are the author's own work:

- **Replication slots.** The primary promises not to recycle WAL a named replica has not consumed. Without them, a replica that falls behind can return to find the WAL it needed is gone and must be rebuilt from scratch. Adding them was the right call.
- **Self-bootstrapping replicas.** The compose command runs `pg_basebackup ... -R` guarded by `if [ ! -s .../PG_VERSION ]`, so a replica builds itself on first boot and skips it on every restart. `-R` writes the standby configuration automatically, which is what makes the copy come up as a replica rather than an independent server. It is idempotent and needs no manual step.

## 4.2 The consequence the project handled well

Async replication means the primary confirms a commit without waiting for replicas. So a read routed to a replica moments after a write can miss the row. The author's framing:

> *"There is no way to route around this without either synchronous replication or read-your-writes logic in the app. I left it async and made the routing honest about it."*

That is correct. The options really are: pay latency (sync), add complexity (pin a user's reads to the primary briefly after their write), or accept staleness and say so. Choosing the third and documenting it is a legitimate decision. Once reads can lag writes, consistency has moved into the application, and that coupling simply does not exist on one node.

## 4.3 Where part 3 is weakest

**"The credential is no longer sitting in version control" is false**

The README's challenge narrative is that a checked-in `.pgpass` was removed and the credential is now generated at container boot from the environment. The change is real and `diff.patch` confirms it. But one `git grep` finds:

```
part_3/replica1.conf:12:primary_conninfo = 'host=primary port=5432 user=postgres password=postgres application_name=replica1'
part_3/replica2.conf:12:primary_conninfo = 'host=primary port=5432 user=postgres password=postgres application_name=replica2'
```

Removing `.pgpass` removed one copy and left two, in the same directory. And `primary_conninfo` governs the *ongoing streaming connection*, which is arguably the more important path than the one-off base backup that `.pgpass` served. The string is also in `.env.example` and `docs/EXPLANATION.md`.
Nothing is at risk (the password is `postgres`, in a lab). But this is checkable in ten seconds. The true and still-good version: *the file whose only purpose was to hold a credential is gone, and the base-backup path now takes it from the environment; the streaming path still has it inline and would need `primary_conninfo` templated at boot the same way.*

**`pg_hba.conf` is `trust` from `0.0.0.0/0`**

`trust` means no password is checked at all: anyone who can open a TCP connection is the `postgres` superuser, from any address, and port 5436 is published to the host. Fine for a disposable lab, but it should be labelled deliberate rather than discovered.
It also creates an internal inconsistency: because replication connections are trusted, `pg_basebackup` never needed a password, so the entire `.pgpass` narrative is defending a door this file leaves open. *(Inferred from what `trust` means and from `primary.conf` loading this file via `hba_file`; not confirmed end to end, since Docker was unavailable.)*

**The tests pass 5/5 with replication completely dead. Verified.**

This was tested, not argued. Writes were pointed at a real database; reads were pointed at a *different, permanently empty* database that receives nothing, ever, which simulates totally broken replication. Result: **5 passed in 1.94s**, with the "replica" holding **0 rows** throughout. Every meaningful assertion sits behind `if len(user_activity) > 0:` or `if type1 in stats_dict:`, which is false forever when replication is broken, so the body never runs. The tests that assert unconditionally only touch the primary.
The suite proves the app can write to the primary and open a connection to something. It does not test replication. The tests are course-provided, the author already identified this in "What I'd do differently", and his proposed fix (poll until the row appears, with a timeout) is exactly right. Stating it first, with the experiment, is a far stronger position than "5/5 passed".

The router's other limits are all self-reported and correct: a new connection per query (no pooling, and PostgreSQL forks a backend process per connection), and `random.choice()` with no health awareness, so a dead replica fails 50% of reads rather than slowing them.

## 5 Verification status

**Docker was unavailable; nothing multi-node was confirmed**

The three Compose stacks were not brought up while writing these documents. No container ran. So **nothing about the topology was verified end to end**: not `pg_basebackup`, not WAL streaming, not `pg_stat_replication` showing two streaming replicas, not the slots, not `pg_hba.conf` being loaded.
Every number in the README's part 1 benchmark table (35 ms → 0.5 ms and the rest) belongs to the original course runs on PostgreSQL 17 in Docker, on another machine. They are **attributed to the README, not verified here**. Where the measurements below disagree with them, they are a *different measurement* on different hardware, a different major version and a different date, not a correction.

What was done: a scratch PostgreSQL 18.4 cluster, loading this repository's own SQL.

| Check | Result |
|---|---|
| `part_1/init.sql` + `solution.sql` | loads; 7 indexes, 36.4 MB total |
| All 14 recovered load-tester queries, planned | every index used by something |
| `status` distribution | 99,000 / 1,000, not "2 values" |
| `COUNT(*)` plan | Index Only Scan, 87 blocks vs 2,494 |
| JSONB GIN at 50% / 17% selectivity | slower / faster |
| `part_2/init.sql` | 600,000 rows; 3,717 in the future |
| Partition row placement | default: 100,000. Weekly: 0, 0, 0 |
| Sequence after migration | `last_value`=1 vs `max(id)`=600,000 |
| Insert after migration | silent duplicate `id`=1 |
| Primary key on `events_partitioned` | none |
| All four part 2 benchmarks | re-measured, both tables analyzed |
| `user_id`-only query | 55 blocks / 1 scan vs 80 blocks / 13 scans |
| part 3 suite, all DSNs to one server | **5 passed** |
| part 3 suite, "replica" permanently empty | **5 passed** (vacuously) |
| `git grep` for credentials | 2 live hits in `replica*.conf` |

# 6 Assessment

## 6.1 The liabilities, worst first

1. "The credential is no longer in version control" is false; one `git grep` disproves it.
2. The 9,154 → 975 headline does not reproduce (measured 236 → 791, i.e. worse). Requalify it to the pruning claim, which does.
3. The user-activity explanation is attached to the wrong query.
4. Migration silently duplicates ids; the partitioned table has no primary key.
5. The weekly "hot data" partitions are empty today.
6. The tests pass vacuously.
7. `idx_users_status` is credited to a query that cannot use it.
8. "Indexes don't help `COUNT(*)`" is wrong by 29x.
9. The "Time Range" benchmark queries a nonexistent user.
10. `PROJECT_WALKTHROUGH.md` makes claims the README carefully does not: "3x read capacity", "200% increase", "70% reduction in slow queries". **Nothing in the repository measures read throughput at all**; no load was ever driven at the replicas. Two documents describing the same project at different confidence levels is itself a liability.

## 6.2 What it gets right, and it is the important part

**It kept a negative result**

Partitioning made a query 70% slower. The author wrote it down, explained it, and drew the correct general conclusion: *partitioning is a bet on your access pattern, and the key choice is really a choice about which queries you are willing to make slower.* That is the best sentence in the repository and it is true.
Alongside it: the conclusion that block counts from `EXPLAIN (BUFFERS)` are a steadier comparison than wall-clock milliseconds, reached independently and correct; the recognition that a first-touch scan is also setting hint bits and writing blocks back out, so a single sample

of it is not a benchmark; and a README that states its own verification gaps rather than implying a live multi-node run happened.
Most portfolio projects report only wins. The defects listed above are all fixable in an afternoon and several belong to the course scaffolding rather than the author. The instinct to publish the loss is not teachable and is worth more than any of them.

### 6.3 Highest-value repairs

1. Fix the credential sentence; template `primary_conninfo` at boot. Ten minutes.
2. Requalify the 9,154 claim to "sixteen partitions pruned to one, `Subplans Removed: 15`". Fifteen minutes, and it becomes reproducible.
3. Generate partitions from the data (`DO` block or `pg_partman`) so the weekly half of the design works again.
4. `setval()` after migration; decide explicitly on `PRIMARY KEY (id, event_time)` and comment why it is forced.
5. Make the tests poll with a timeout so 5/5 means something.
6. Reconcile or delete `PROJECT_WALKTHROUGH.md`.

## 7 Glossary

**Block (page)** The 8 KB unit of I/O. PostgreSQL never reads half of one. Block counts are the honest measure of query work; milliseconds move with cache warmth.

**Heap** The unordered file holding a table's rows.

**Sequential scan** Read every block, test every row. Costs the same whether one row matches or all of them.

**Index** A separate sorted structure mapping values to row locations. Not free: every write must update every index on the table.

**B-tree** The default index. A shallow balanced tree, so a lookup is a handful of block reads instead of thousands.

**GIN** Generalized Inverted Index. Maps one *component* to many rows. The right structure when one row contains many searchable things (trigrams, JSON keys). Returns candidates that must be re-checked.

**Operator class** The rulebook telling an index how to compare values, and therefore which operators it can answer. `text_pattern_ops` is one.

**pg_trgm / trigram** Extension that chops strings into three-character slices so GIN can serve `LIKE '%x%'` and `ILIKE`.

**JSONB / `@>`** Binary JSON column type; `@>` is "contains", the operator GIN can accelerate.

**Query planner** Picks the strategy by estimating costs from statistics. It can rationally refuse to use your index, and it can be wrong.

**ANALYZE** Refreshes the planner's statistics. Its absence is the most likely explanation for this project's unreproducible headline number.

**Index Only Scan** Answering entirely from an index without touching the heap. Requires an up-to-date visibility map, which is maintained by `VACUUM`.

**Selectivity** The fraction of rows a predicate matches. Not the number of distinct values in the column. Confusing the two is the root of this project's status-index error.

**EXPLAIN (ANALYZE, BUFFERS)** Run the query and report the real plan, real row counts, real timings and blocks touched. The core skill part 1 teaches.

**Hint bit** A per-row note of whether its creating transaction committed. The first reader sets it, which is why a first-touch scan is also a large write and why a single sample of it is not a benchmark.

**Partitioning / partition key** Splitting a table into physical children by a rule on a column. The key is the column the rule uses.

**Partition pruning** The planner proving a partition cannot match and skipping it. Visible as `Subplans Removed: N`.

**WAL** Write-Ahead Log. Every change is logged before data is touched, for crash safety. Replication is crash recovery aimed at another machine.

**Primary / replica** The primary takes writes; replicas stream and replay its WAL and serve read-only queries.

**Async replication** The primary does not wait for replicas before confirming a commit. Fast; reads can be stale.

**Replication slot** The primary's promise not to recycle WAL a named replica has not consumed yet.

**pg_basebackup** Byte-level physical copy of a whole data directory; how a replica gets its starting point. `-R` writes the standby config so the copy comes up as a replica.

**DSN** Connection string packing host, port, database, user and password into one URL.

**Write amplification** One logical write becoming many physical ones, because every index must be updated too. The unmeasured cost of part 1.

---

**The one-paragraph answer**

A CTO wanted to migrate to NoSQL because queries were slow. This project works through the standard PostgreSQL toolbox instead. The right index turns a 2,494-block sequential scan into a 4-block lookup, but only if you pick the right operator class: a default B-tree cannot answer a prefix match, a substring match, or a JSON containment query at all. Partitioning then reorganizes a time-series table so a March query touches only March, verified by the planner pruning fifteen of sixteen partitions; but it is a bet on the access pattern, and a query filtering on anything except the partition key gets slower, which this project measured and kept. Finally, streaming replication adds read capacity by shipping the write-ahead log to two copies, and pushes consistency into the application: reads can now be stale, so the app owns routing and the tests own the lag. None of it required leaving PostgreSQL.